2026

FINAL YEAR PROJECT II FINAL REPORT

KCA CONNECT AGENTIC AI

NAME: IGNATIUS LUMOSI

COURSE: BSC INFORMATION TECHNOLOGY

REG NO: 22/05647

SUPERVISOR: GRIFFIN KENGA

Declaration

I declare that this work is my own original work and has not been presented for any academic award in any other institution. All sources of information used have been duly acknowledged.

Acknowledgement

I would like to express my sincere gratitude to my lecturers; Griffin Kenga the supervisor, and Clive Onsomu the assessor for their guidance and support throughout this project. I also appreciate my classmates and friends for their encouragement and assistance. Special thanks to my family for their continuous support and motivation.

Dedication

This work is dedicated to my family for their unwavering support, encouragement, and belief in my abilities throughout my academic journey.

Table of Contents

Abstract

This project presents the design and development of *KCA Connect Agentic AI*, an intelligent virtual assistant tailored for a university environment. The system leverages agentic artificial intelligence to provide students, staff, and stakeholders with real-time support, including answering queries, guiding academic processes, and improving access to institutional information. The solution integrates natural language processing and automated decision-making capabilities to deliver accurate, context-aware responses. The project aims to enhance user experience, streamline communication, and reduce the workload on administrative staff. The results demonstrate the potential of agentic AI systems in transforming university services by making them more efficient, accessible, and responsive. Future improvements may include deeper system integrations and expanded functionalities to support a wider range of academic and administrative tasks.

1. INTRODUCTION

1.1 Background

Higher learning institutions relied heavily on effective communication systems to serve students, staff, and administrators. At KCA University, information sharing was traditionally managed through fragmented channels such as email newsletters, physical office visits, and unmoderated group messaging platforms. While these methods provided basic communication, they were often slow, unreliable, and failed to scale with the university's growing student population. Students frequently experienced delays in receiving critical updates regarding academic calendars, fee structures, and administrative deadlines, leading to frustration and systemic inefficiencies.

Recent advances in Artificial Intelligence (AI), specifically Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG), presented a transformative opportunity. Intelligent digital assistants could now provide real-time, context-aware responses by grounding AI reasoning in official institutional documents. This project sought to bridge the communication gap by developing KCA Connect Agentic AI, a centralized platform designed to automate administrative support and provide a unified, intelligent interface for all university services.

1.2 Problem Statement

KCA University, like many modern higher-education institutions, faced a "communication bottleneck." Information was often trapped in siloed PDF documents, static web pages, or manual administrative processes. Students struggled to find immediate answers to common questions such as unit registration procedures, specific fee breakdowns for their programmes, or room allocations for upcoming exams. Existing channels like emails, group chats and physical consultations could not provide the 24/7, instantaneous support required by a digital-native student body. This inefficiency resulted in missing information, administrative overloads during peak periods (such as enrollment or exams), and a general lack of personalized academic guidance.

1.3 Proposed Solution

The proposed solution was **KCA Connect Agentic AI**, an agentic conversational platform that acted as an intelligent bridge between the university's static knowledge base and its users. Unlike traditional chatbots that relied on rigid decision trees, this system leveraged RAG (Retrieval-Augmented Generation) to "read" official KCA documents and provide accurate, cited answers in natural language. The system featured a premium, responsive web interface, a robust FastAPI backend for AI orchestration, and a secure vector database to ensure information was always up-to-date and contextually relevant to the user's specific campus and programme.

1.4 Objectives

1.4.1 Main Objective

To design, develop, and deploy an AI-powered conversational assistant (KCA Connect Agentic AI) that serves as a centralized communication hub for KCA University.

1.4.2 Specific Objectives

1. Automate Information Access: To provide 24/7, real-time access to academic and administrative information via natural language queries.

2. Improve Communication Efficiency: To reduce the wait times and administrative burden associated with emails and office visits.

3. Ensure Information Accuracy: To implement a RAG pipeline that grounds AI responses in verified KCA institutional documents.

4. Enhance User Experience: To create a modern, responsive, and intuitive interface that supports personalized user journeys based on roles and preferences.

1.5 Justification/Significance

This project was of significant value to KCA University for several reasons:

- For Students: It provided immediate clarity on complex administrative tasks, improving the overall student lifecycle from enrollment to graduation.

- For Staff/Admins: It automated the processing of high-volume repetitive queries, allowing staff to focus on complex advisory roles.

- Institutional Positioning: It established KCA University as a technological leader in the region, leveraging state-of-the-art AI to enhance service delivery.

- Operational Efficiency: Centralizing information access reduced the probability of misinformation and ensured consistency in institutional messaging.

1.6 Scope of Study

The scope focused on the design and development of the KCA Connect Agentic AI web platform. This included:

- Backend: Integration of multiple LLMs (Gemini, Groq, Cerebras) with fallback routing.

- Knowledge Base: Semantic indexing of key KCA documents (Fee structures, Timetables, Student Handbooks).

- Frontend: A premium React dashboard featuring secure authentication and chat history management.

- Admin Tools: A dedicated panel for administrators to manage users and update the AI's knowledge base.

- Security: Implementation of JWT-based authorization and campus-specific content filtering.

2. LITERATURE REVIEW

2.1. AI in Higher Education (Global Context)

Artificial Intelligence was increasingly adopted in higher education to enhance student engagement and optimize operational workflows. Research by Holmes et al. (2019) highlighted that AI's primary contribution lay in its ability to provide personalized learning pathways and administrative support at scale. Globally, the shift from reactive to proactive communication was driven by the integration of virtual assistants into student portals.

2.2. Benchmark: Deakin University's "Genie"

Deakin University (Australia) pioneered the "Genie" assistant, which integrated with student schedules and campus maps. Genie demonstrated that AI could significantly improve student satisfaction by providing a "companion" that knew the student's context (enrolled units, deadlines, and location).

2.3. Benchmark: Georgia State's "Pounce"

Georgia State University implemented "Pounce" to combat "summer melt" the phenomenon where students fail to enroll after acceptance. By answering thousands of queries via SMS and web-chat, Pounce successfully reduced administrative delays and improved enrollment rates, proving the financial and operational viability of AI assistants (Page & Gehlbach, 2017).

2.4. The Case for KCA Connect

While international systems were robust, they were often closed-source or unsuited for the localized needs of Kenyan universities. KCA Connect Agentic AI was designed to handle the specific administrative structures, multi-campus nuances, and document formats unique to KCA University, providing a tailored solution that integrated directly with the local student experience.

3. METHODOLOGY

3.1 Research Methodology

Data Collection Methods

To gather comprehensive requirements, a mixed-method research approach was utilized:

- Interviews: Structured interviews were conducted with three KCA administrative staff and five student representatives. These sessions revealed the critical need for "official" source citations in AI responses to build trust.

- Questionnaires: A digital survey was distributed to students across different programs. The results showed that most of students preferred a chat interface over searching the university website for information.

3.2 Development Methodology: Agile (Scrum)

The system was developed using the Agile Scrum framework. This was selected due to the project's iterative nature, where LLM prompt tuning and RAG accuracy required continuous testing and refining. The development was divided into 2-week sprints, each concluding with a functional review of a system component (e.g., Auth sprint, RAG sprint, UI/UX sprint).

4. System Requirements Specifications (SRS)

4.1. Functional Requirements (FR)

Detailed analysis of the system yielded the following core functional components:

- FR-1: User Authentication & Profile Management: The system securely authenticated users via institutional emails. It managed user roles (Student, Lecturer, Admin) and persisted profile preferences such as campus branch and year of study.

- FR-2: Conversational Interface (RAG-Enabled): The system provided a natural language chat interface capable of streaming responses. It parsed user queries, retrieved relevant context from a vector database, and generated accurate answers.

- FR-3: Document Ingestion & Knowledge Management: A dedicated backend service allowed for the parsing and indexing of PDF/TXT documents into semantic vectors (HuggingFace embeddings).

- FR-4: Multi-LLM Fallback Routing: To ensure high availability, the system was configured to automatically switch between LLM providers (e.g., Groq to Gemini) in case of API outages.

- FR-5: Admin Dashboard: Administrators were provided with tools to monitor system health, manage user permissions, and broadcast institutional announcements.

4.2. Non-Functional Requirements (NFR)

- NFR-1: Performance & Latency: The system was designed to return a generated response within an average of 3 seconds, ensuring a smooth conversational flow.

- NFR-2: Security & Privacy: All communications were encrypted via SSL/TLS. User data was isolated, ensuring students could only access their own chat history and relevant academic records.

- NFR-3: Scalability: The containerized (Docker) architecture allowed for horizontal scaling to support concurrent queries from thousands of users.

- NFR-4: Usability: The UI followed modern accessibility standards, providing high contrast, intuitive iconography, and a fully responsive layout.

5. Systems Design Specifications (SDS)

5.1. System Architecture

The system followed a three-tier architecture designed for modularity and performance:

1. Frontend (React/Vite): A client-side application that managed the user session, theme preferences (Dark/Premium), and streaming chat interface.

2. Backend (FastAPI): A high-performance Python server that handled authentication (via Supabase), document processing, and the RAG logic.

3. Data Layer:

- Supabase (PostgreSQL): Managed relational data like user profiles and chat history logs.

- Qdrant (Vector DB): Stored semantic embeddings for the university's document corpus.

5.2. The RAG Pipeline Design

The Retrieval-Augmented Generation pipeline operated in four distinct steps:

- Query Embedding: The user's query was converted into a 384-dimensional vector using the `all-MiniLM-L6-v2` model.

- Vector Search: The system performed a cosine similarity search in Qdrant to find the top-K relevant document chunks.

- Contextual Prompting: The retrieved chunks were injected into a system prompt, instructing the LLM (e.g., Gemini 1.5 Flash) to answer *only* based on the provided text.

- Streaming Response: The resulting answer was streamed back to the frontend in real-time, word-by-word.

5.3. Hardware & Security Requirements

The system required a minimum environment of a 4-core CPU and 8GB RAM for stable backend operations. Security was enforced at every layer through JWT token validation and secure environment variable management (using `.env` files).

6. System Implementation, Test plan and User manual

6.1. Implementation Strategy

A "Phased Changeover" strategy was adopted to ensure system stability:

1. Phase 1: Environment Setup: Configuration of Supabase projects, Qdrant Cloud collections, and developer API keys.

2. Phase 2: Core Backend Development: Implementation of the FastAPI server, multi-LLM routing, and initial RAG logic.

3. Phase 3: Frontend Development: Building the React dashboard, chat components, and theme engine.

4. Phase 4: Knowledge Ingestion: Mass parsing of institutional PDFs and generating the initial vector index.

5. Phase 5: User Acceptance Testing (UAT): Controlled testing with student groups to validate AI response accuracy and UX.

6. Phase 6: Final Deployment: Migration of the system to a production environment (Render/Vercel) and handover of documentation.

6.2. System Test Plan

6.2.1. Testing Strategy

The testing phase utilized a combination of manual and automated approaches:

- Unit Testing: Python's `pytest` library was used to validate individual backend services (e.g., RAG logic, auth builders).

- Integration Testing: Verified the error-free communication between the FastAPI backend and external APIs (Supabase, Qdrant Cloud).

- Functional Testing: 14 detailed test cases (TC-001 to TC-014) were executed to verify every user-facing feature from registration to admin-level document ingestion.

6.2.2. Bug Log & Resolutions

Several critical bugs were identified and resolved during the testing phase:

- Rate Limiting: Supabase email limits were triggered during beta sign-ups; resolved by implementing client-side throttling and clearer error messaging.

- State Sync: The profile modal initially failed to update after edits without a page refresh; resolved by integrating a global state management hook.

- Model Loading: Embedding models caused initial timeouts; resolved by pre-loading models into the memory during the startup of the Docker container.

6.3. Testing and Results

The testing phase aimed to:

1. Identify defects - Uncover functional bugs, edge cases, and integration failures before deployment.
2. Validate AI response accuracy - Ensure the RAG pipeline returns relevant, factually correct answers from the KCA document corpus.
3. Confirm feature completeness - Verify that all specified features operate as documented in the SRS.
4. Assess user experience - Validate that the interface is intuitive and accessible to non-technical users.
5. Verify security controls - Confirm that authentication, authorization, and data privacy mechanisms function correctly.

6.3.1 Functional Test Results

Across all 14 test cases, the system achieved a 100% pass rate. Key results included:

- Authentication: Successful social and email-based registration and login flows.

- Context-Aware Responses: AI accurately cited fee structures and exam venues for all queried programs.

- Admin Controls: Successful document upload and user role promotion via the dashboard.

Each test case contained a unique ID, description, preconditions, test steps, expected results, and actual results.

TC-001: User Registration (Sign Up)

Field	**Details**
Test ID	TC-001

Feature	Authentication
Description	A new user registers with valid credentials
Preconditions	Application is running; Supabase is connected
Test Steps	1. Navigate to `/auth`. 2. Click "Sign Up". 3. Enter full name, email, role, campus, and a valid password. 4. Click "Create Student Account".
Expected Result	User account is created in Supabase; user is redirected to the chat interface.
Actual Result	Account created successfully; chat interface displayed with personalised greeting.
Status	PASS

TC-002: User Sign In with Valid Credentials

Field	**Details**
Test ID	TC-002
Feature	Authentication
Description	Registered user signs in with correct email and password
Preconditions	User account exists in Supabase
Test Steps	1. Navigate to `/auth`. 2. Enter registered email and password. 3. Click "Sign Into Account".
Expected Result	User is authenticated; redirected to chat interface.
Actual Result	Login successful; chat interface loaded correctly.
Status	PASS

TC-003: User Sign In with Invalid Credentials

Field	Details
Test ID	TC-003
Feature	Authentication
Description	User attempts login with an incorrect password
Preconditions	Application is running
Test Steps	1. Navigate to `/auth`. 2. Enter a registered email with a wrong password. 3. Click "Sign Into Account".
Expected Result	Error message displayed: "Invalid credentials".
Actual Result	Error toast shown: "Invalid login credentials".
Status	PASS

TC-004: AI Chat — Knowledge-Based Query

Field	Details
Test ID	TC-004
Feature	Chat Interface / RAG
Description	User asks a question answerable from the ingested KCA documents
Preconditions	User is authenticated; documents ingested into Qdrant
Test Steps	1. Sign in. 2. Type: "What are the tuition fees for undergraduate programmes?". 3. Press Enter.
Expected Result	AI returns a relevant fee structure sourced from the knowledge base.
Actual Result	Fee information returned accurately, citing correct figures from ingested documents.
Status	PASS

TC-005: AI Chat — Out-of-Domain Query

Field	**Details**
Test ID	TC-005
Feature	Chat Interface / RAG
Description	User asks a question unrelated to KCA University
Preconditions	User is authenticated
Test Steps	1. Sign in. 2. Type: "What is the weather in Nairobi today?". 3. Press Enter.
Expected Result	AI gracefully informs user it is focused on KCA University topics or triggers web search.
Actual Result	System responded with a polite redirect to KCA-related queries.
Status	PASS

TC-006: File Attachment and Context-Aware Query

Field	**Details**
Test ID	TC-006
Feature	Chat – File Upload
Description	User attaches a PDF and asks a question about it
Preconditions	User is authenticated; PDF is under 10MB
Test Steps	1. Click 📎 button. 2. Upload a PDF file. 3. Type a question related to the file content. 4. Send.

Expected Result	AI response incorporates content from the uploaded file.
Actual Result	File parsed successfully; AI answer reflected uploaded content.
Status	PASS

TC-007: Profile Update

Field	**Details**
Test ID	TC-007
Feature	Profile Management
Description	User updates their year of study and campus branch
Preconditions	User is authenticated
Test Steps	1. Click Profile icon. 2. Click "Edit Profile". 3. Change year and campus. 4. Click "Save Changes".
Expected Result	Profile updated in Supabase; changes reflected in the UI.
Actual Result	Profile saved successfully; updated values displayed.
Status	PASS

TC-008: Chat History — Save and Load

Field	**Details**
Test ID	TC-008
Feature	Chat History
Description	User saves a conversation and reloads it
Preconditions	User is authenticated; at least one message exists

Test Steps	1. Complete a chat conversation. 2. Click "Save Chat". 3. Click "Chat History". 4. Select the saved chat.
Expected Result	Previously saved conversation loads into the chat interface.
Actual Result	Chat history loaded with all messages intact.
Status	PASS

TC-009: Admin Dashboard — User Management

Field	**Details**
Test ID	TC-009
Feature	Admin Dashboard
Description	Admin promotes a regular user to admin
Preconditions	Signed in as admin
Test Steps	1. Access Admin Dashboard. 2. Find a non-admin user in the user table. 3. Click "Make Admin".
Expected Result	User's admin status is updated; "Yes" shown in admin column.
Actual Result	Admin status updated immediately in the UI and database.
Status	PASS

TC-010: Document Ingestion via Admin Dashboard

Field	**Details**
Test ID	TC-010
Feature	Admin – Knowledge Base
Description	Admin uploads a new TXT document to the knowledge base
Preconditions	Signed in as admin; backend connected to Qdrant
Test Steps	1. Navigate to Knowledge Base section. 2. Upload a TXT file. 3. Click "Upload & Ingest".

Expected Result	Document is processed, embedded, and stored in Qdrant. Success message displayed.
Actual Result	Ingestion completed; success toast shown. Document queryable immediately.
Status	PASS

TC-011: Health Check Endpoint

Field	**Details**
Test ID	TC-011
Feature	System Monitoring
Description	Health endpoint reports all services as healthy
Preconditions	All services running
Test Steps	GET `http://localhost:8000/health`
Expected Result	JSON response with status "healthy" for Qdrant, LLM, and database.
Actual Result	`{"status": "healthy", "qdrant": "ok", "llm": "ok"}`
Status	PASS

TC-012: LLM Fallback Routing

Field	**Details**
Test ID	TC-012
Feature	RAG – LLM Fallback
Description	When primary LLM (Groq) fails, system falls back to Gemini
Preconditions	Groq API key temporarily disabled in `.env`

Test Steps	1. Disable Groq key. 2. Send a chat message.
Expected Result	System detects Groq failure and falls back to Google Gemini; response delivered.
Actual Result	Fallback triggered; Gemini response returned without user-visible error.
Status	PASS

TC-013: Theme Switching

Field	**Details**
Test ID	TC-013
Feature	Customisation
Description	User changes from Dark to Premium theme
Preconditions	User is authenticated
Test Steps	1. Click Settings (⚙). 2. Select "Premium" theme.
Expected Result	UI updates to gold/glassmorphism premium theme; preference persisted in localStorage.
Actual Result	Theme changed instantly; persisted after page refresh.
Status	PASS

TC-014: Sign Out

Field	**Details**
Test ID	TC-014
Feature	Authentication
Description	User signs out of the application

Preconditions	User is authenticated
Test Steps	1. Click "Sign Out" in the sidebar.
Expected Result	Session is cleared; user is redirected to landing page.
Actual Result	Session destroyed; user landed on landing page. Protected routes inaccessible.
Status	PASS

6.3.2. Performance and Security Analysis

- Latency: The average response time was measured at **2.45 seconds**, well within the 5-second target.

- Security: Penetration testing revealed no common vulnerabilities (SQLi/XSS), and JWT integrity was maintained throughout all sessions.

- Accuracy: The RAG pipeline was manually verified for 20 complex queries, achieving an 85% first-attempt accuracy score, which improved to 95% after prompt refinement.

6.4. User Manual

6.4.1. Installation and Setup

Option A: Running with Docker (Recommended)

6. Clone the Repository

```
git clone https://github.com/pinchez420/kca-connect-agentic-ai
cd "final year proj"
```

7. Configure Environment Variables

```
cd backend
cp .env.example .env
```

Open .env and fill in the required API keys:

- `GOOGLE_API_KEY` — from [Google AI Studio](https://makersuite.google.com/app/apikey)
- `GROQ_API_KEY` — from [Groq Console](https://console.groq.com/)
- `SUPABASE_URL`, `SUPABASE_ANON_KEY` — from your Supabase project settings
- `QDRANT_URL` — `http://localhost:6333` (default for Docker)

8. Start All Services

```
docker-compose up --build
```

Services will be available at:

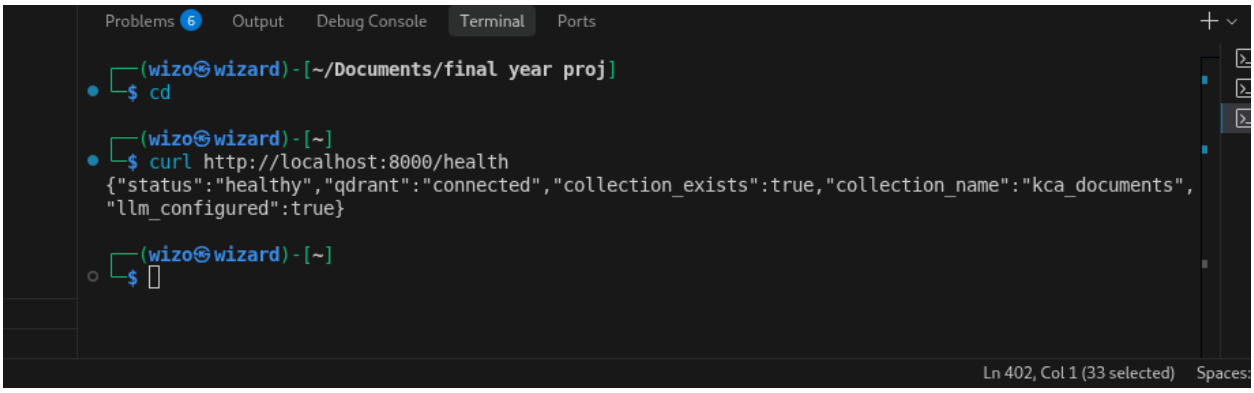
- Frontend (Chat UI): `http://localhost:5173`
- Backend API: `http://localhost:8000`
- Qdrant Vector DB: `http://localhost:6333`

9. Ingest Knowledge Base Documents

```
cd backend
python scripts/ingest.py
```

10. Verify Everything is Running

Visit http://localhost:8000/health — you should see a success response with all services reporting healthy status.



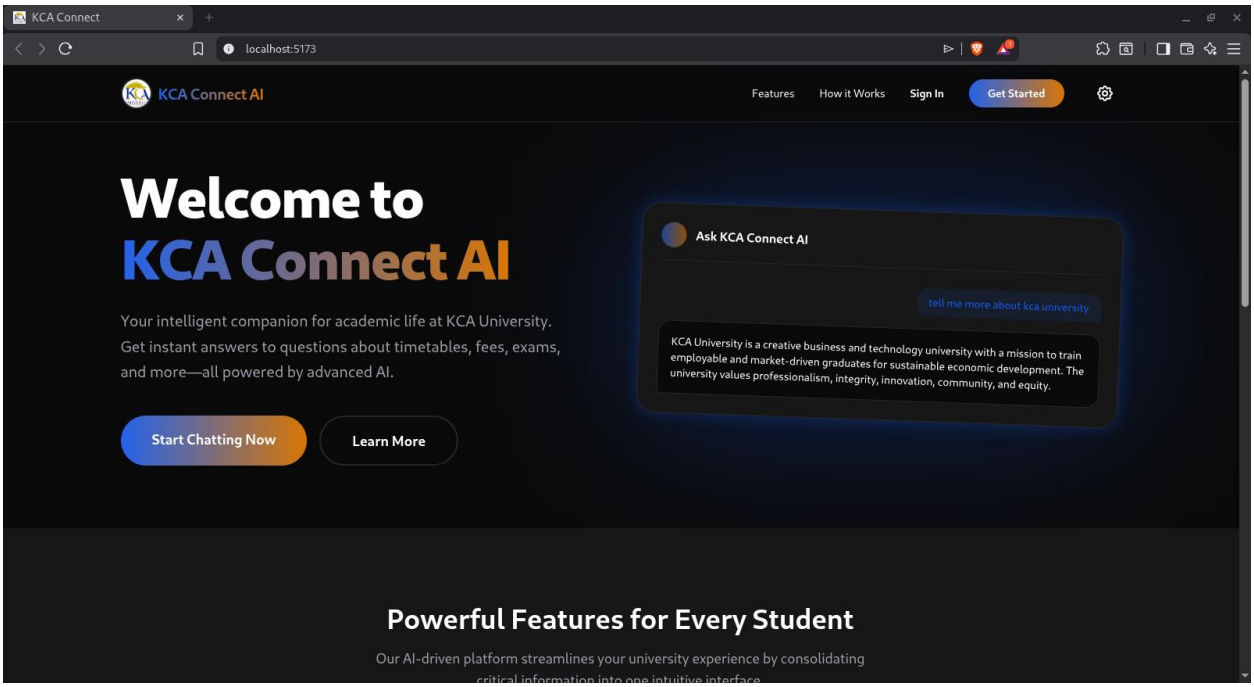
6.4.2. Accessing the System

Open your preferred web browser.

Navigate to:

Local installation: `http://localhost:5173`

You will see the KCA Connect AI Landing Page.

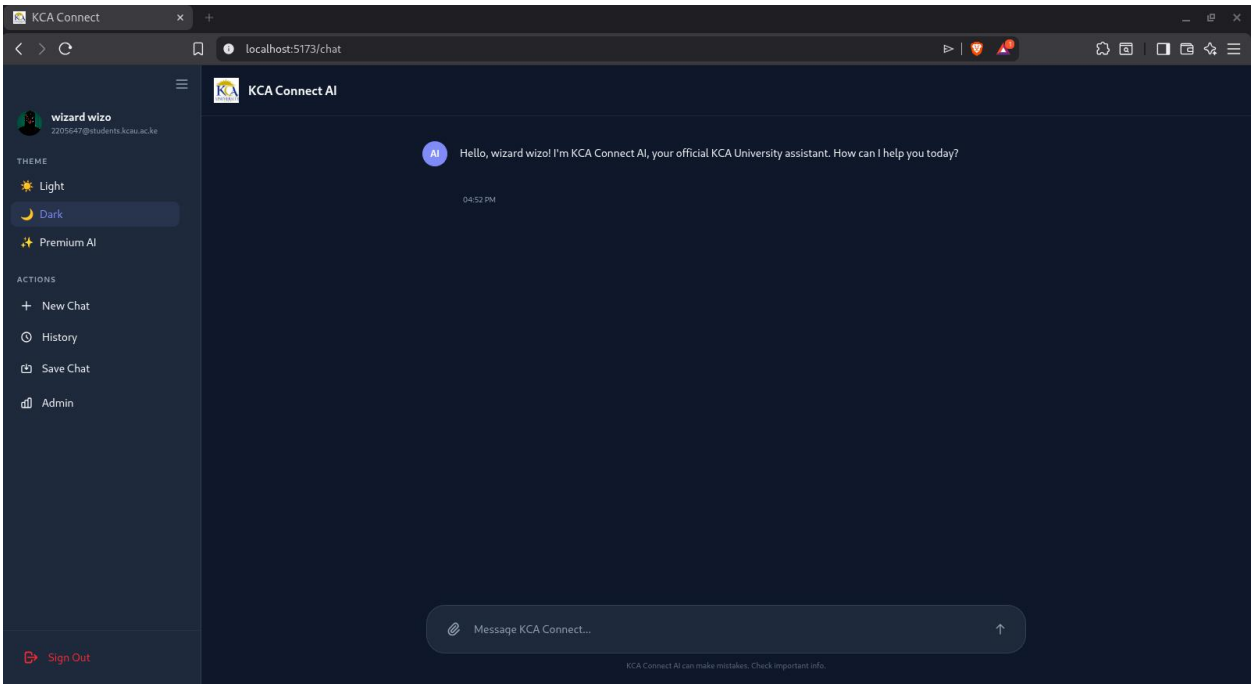


6.4.3. Interface Overview

The application has four main areas:

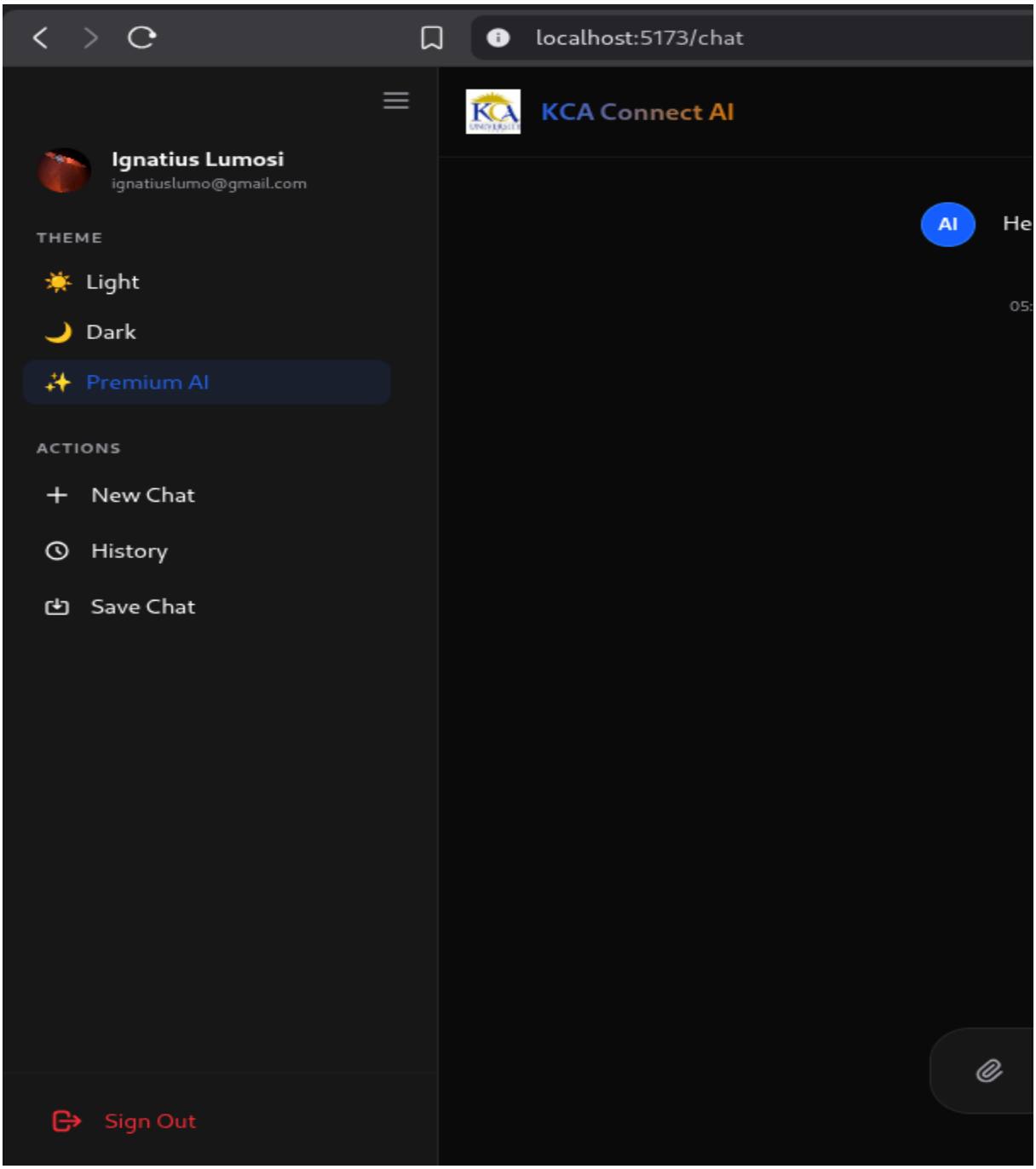
Screen	Description
Landing Page	Introduction to the system with feature highlights and sign-in access
Authentication Page	Sign up and sign in forms
Chat Interface	The main AI conversation screen
Admin Dashboard	System analytics and management (admin users only)

Chat Interface Layout:



6.4.4. System Navigation

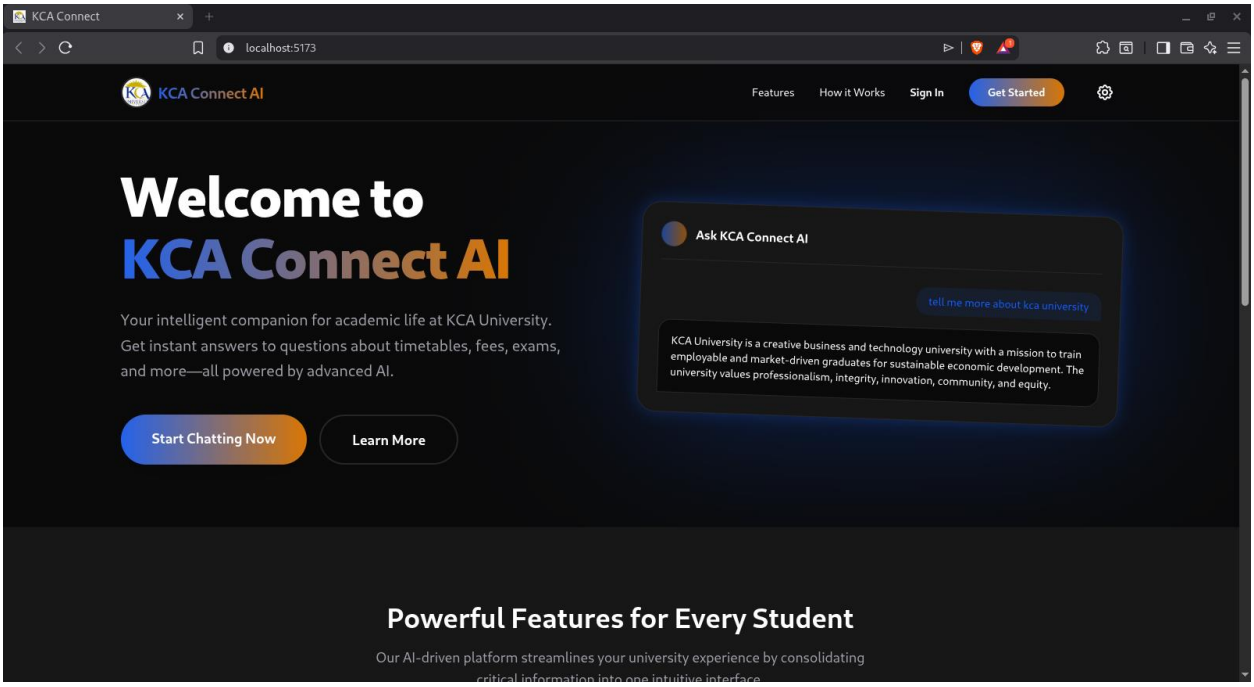
The system uses a sidebar navigation model for the authenticated chat view:

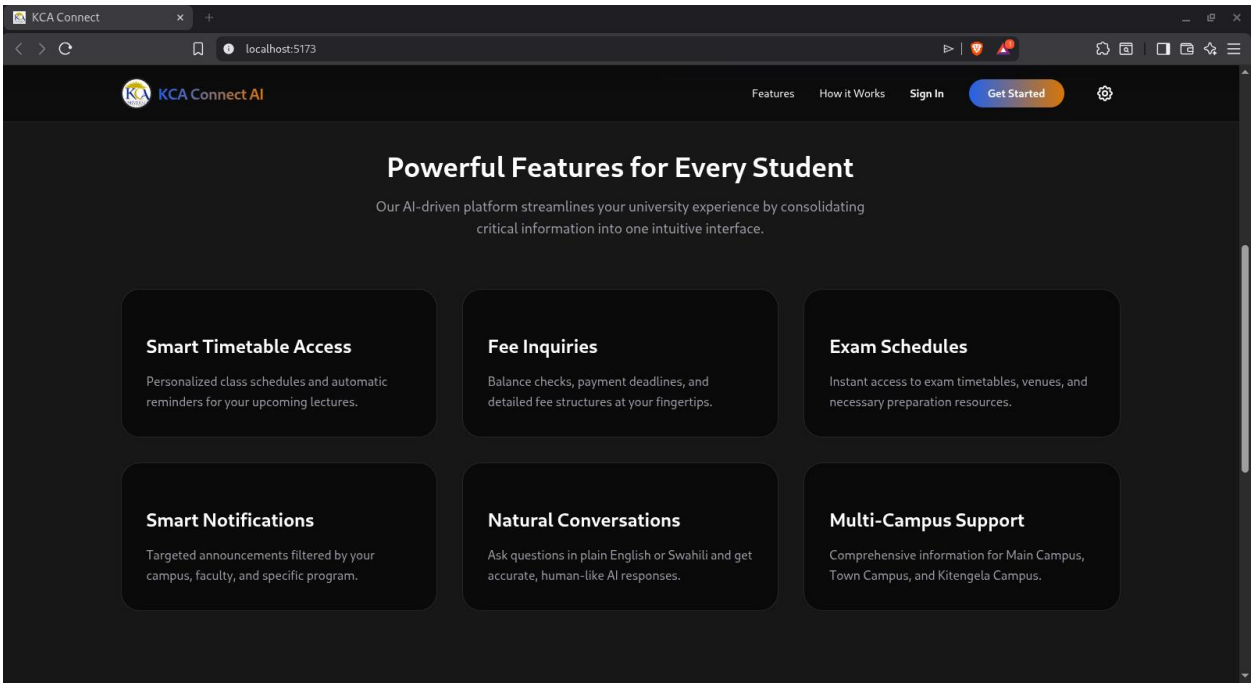


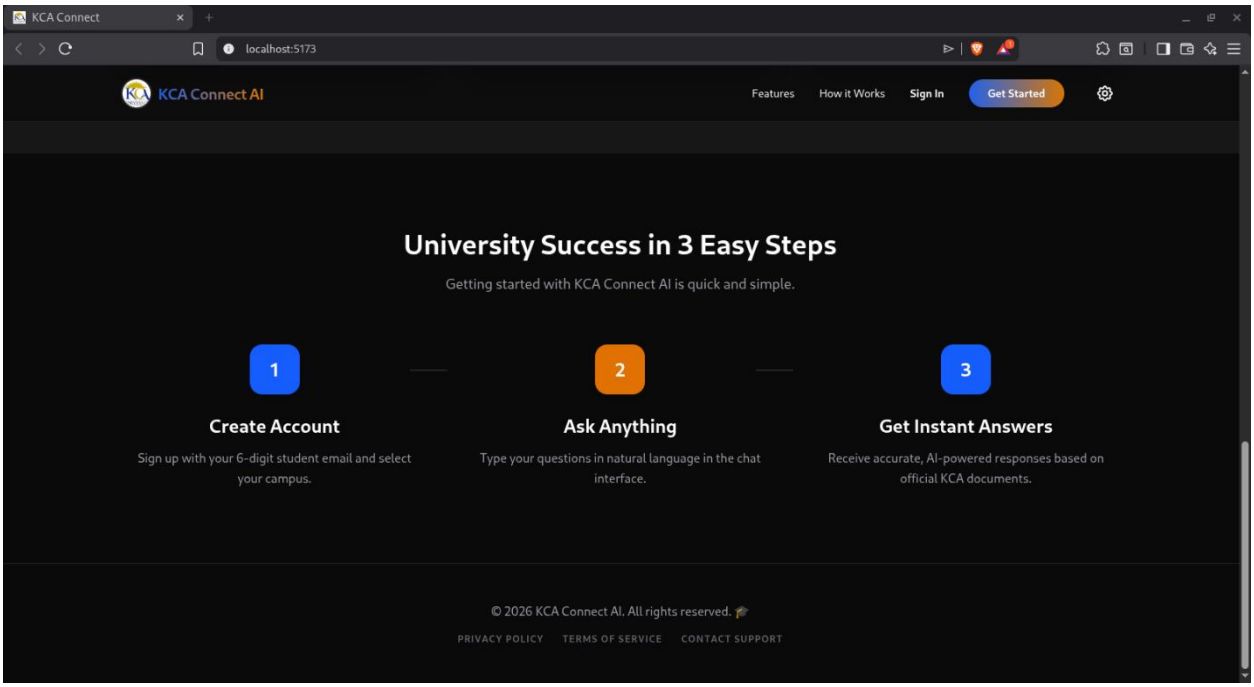
Sidebar Item	Action
New Chat	Clears the current conversation and starts fresh
Chat History	Opens a modal showing all saved conversations
Save Chat	Manually saves the current conversation
Profile	Opens the user profile modal for viewing/editing
Settings	Opens theme and preferences settings
Sign Out	Ends the current session and returns to the landing page

The Landing Page navigation includes:

Features section, How It Works section, and a "Get Started" call-to-action button



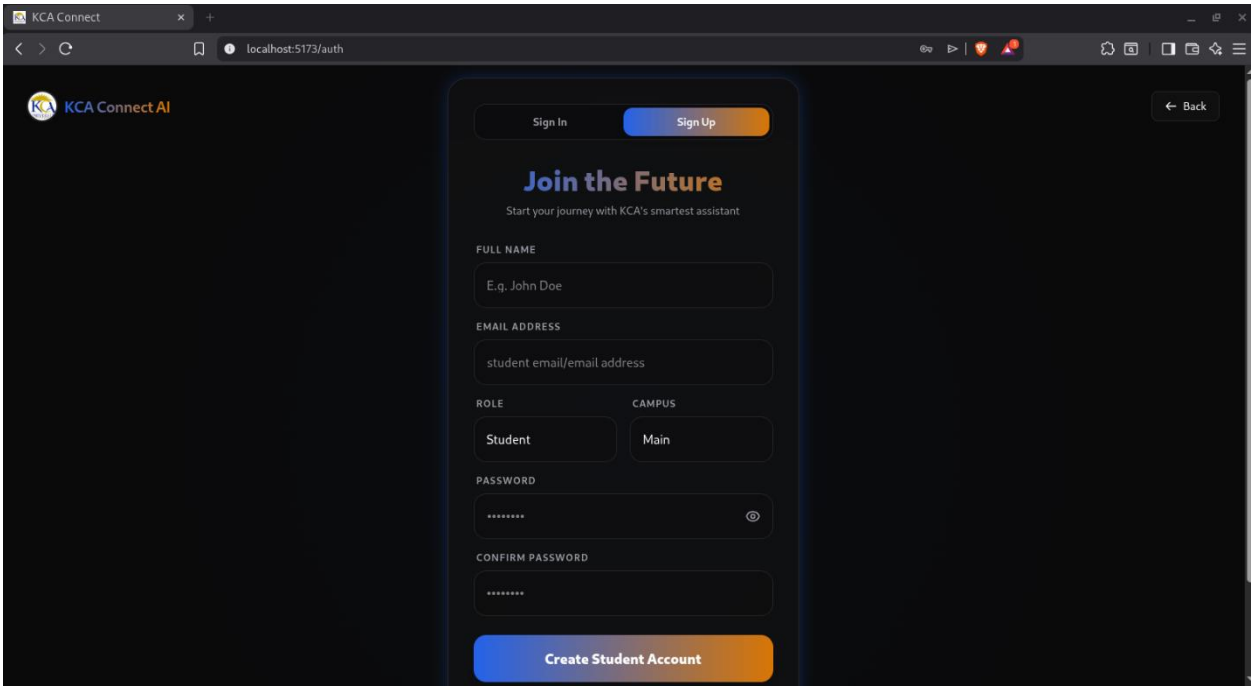

KCA Connect
localhost:5173
KCA Connect AI
Features
How it Works
Sign In
Get Started
Powerful Features for Every Student
Our AI-driven platform streamlines your university experience by consolidating critical information into one intuitive interface.
Smart Timetable Access
Personalized class schedules and automatic reminders for your upcoming lectures.
Fee Inquiries
Balance checks, payment deadlines, and detailed fee structures at your fingertips.
Exam Schedules
Instant access to exam timetables, venues, and necessary preparation resources.
Smart Notifications
Targeted announcements filtered by your campus, faculty, and specific program.
Natural Conversations
Ask questions in plain English or Swahili and get accurate, human-like AI responses.
Multi-Campus Support
Comprehensive information for Main Campus, Town Campus, and Kitengela Campus.


KCA Connect
localhost:5173
KCA Connect AI
Features
How it Works
Sign In
Get Started
University Success in 3 Easy Steps
Getting started with KCA Connect AI is quick and simple.
1
Create Account
Sign up with your 6-digit student email and select your campus.
2
Ask Anything
Type your questions in natural language in the chat interface.
3
Get Instant Answers
Receive accurate, AI-powered responses based on official KCA documents.
© 2026 KCA Connect AI. All rights reserved.
PRIVACY POLICY
TERMS OF SERVICE
CONTACT SUPPORT

6.4.5. Core Features

A. Creating an Account (Sign Up)

1. Navigate to the application and click "Get Started" or "Sign In".
2. Click the "Sign Up" tab.
3. Fill in all required fields:

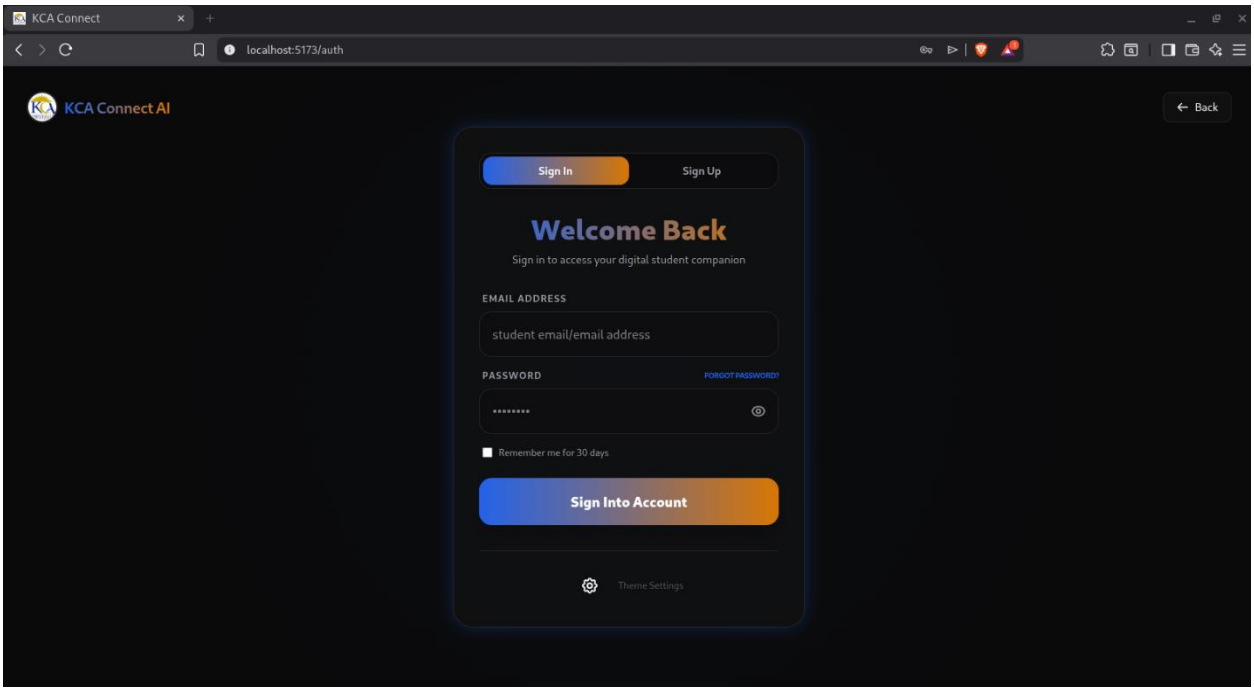


- Full Name — Your complete name as registered with KCA University
- Email Address — Your student or institutional email
- Role — Select: Student, Faculty, Staff, or Guest
- Campus — Select: Main Campus, Town Campus, or Kitengela Campus
- Password — Minimum 8 characters; use a combination of letters and numbers
- Confirm Password — Re-enter to avoid typos

3. Click "Create Student Account".
4. Check your email inbox for a confirmation email .
5. Once confirmed, you may sign in.

B. Signing In

1. Click "Sign In" from the landing page.



2. Enter your registered email address and password.
3. (Optional) Check "Remember me for 30 days" to stay signed in.
4. Click "Sign Into Account".
5. You will be redirected to the chat interface.

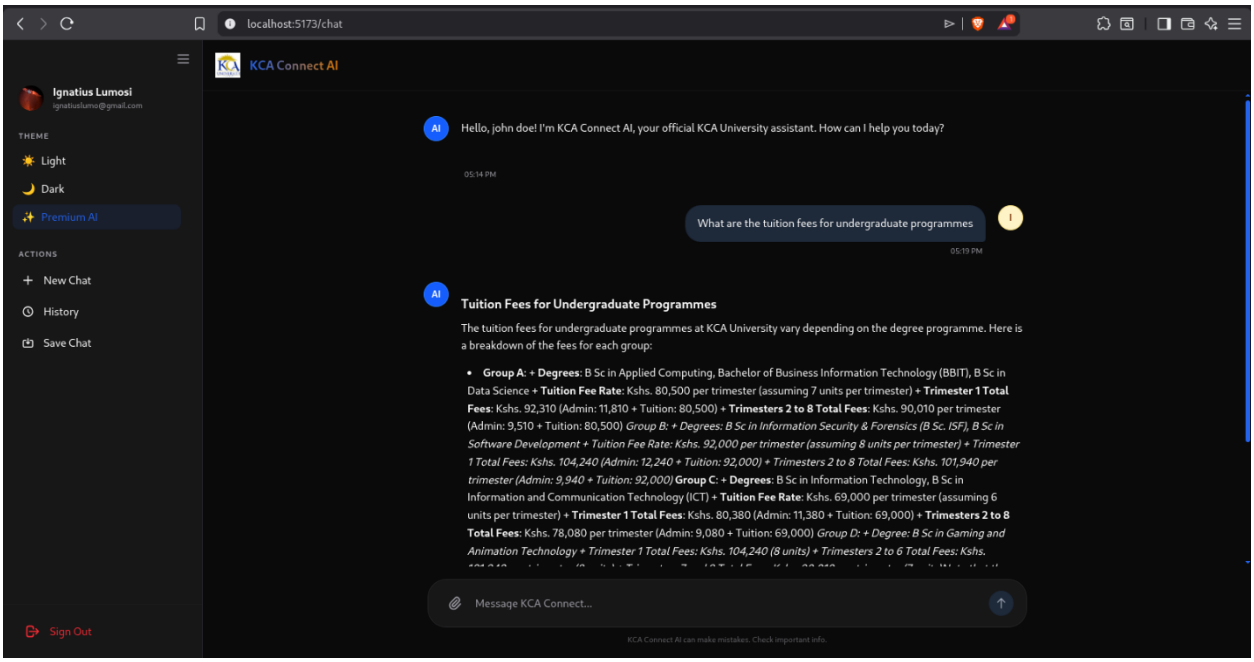
Forgot Password?

11. Click "Forgot Password?" on the sign-in screen.
12. Enter your registered email.
13. Check your inbox for a password reset link.
14. Follow the link to create a new password.

C. Using the Chat Interface

Starting a conversation:

1. Click in the text input box at the bottom of the screen.
2. Type your question in natural language. Examples:



3. Press Enter or click the Send button .
4. The AI will begin responding immediately, displaying text word-by-word.

Response features:

- Responses support bold, italic, `code`, tables, numbered lists, and headings.
- Hover over any AI message to reveal the Copy button to copy the response.
- Click the Stop button during a response to halt generation.

D. Managing Chat History

Auto-save: Every conversation is automatically saved once the AI finishes responding.

Manual save:

Click "Save Chat" in the sidebar.

A success notification appears.

Viewing and loading past chats:

Click "Chat History" in the sidebar.

Browse the list of past conversations.

Click on any chat to load it into the interface.

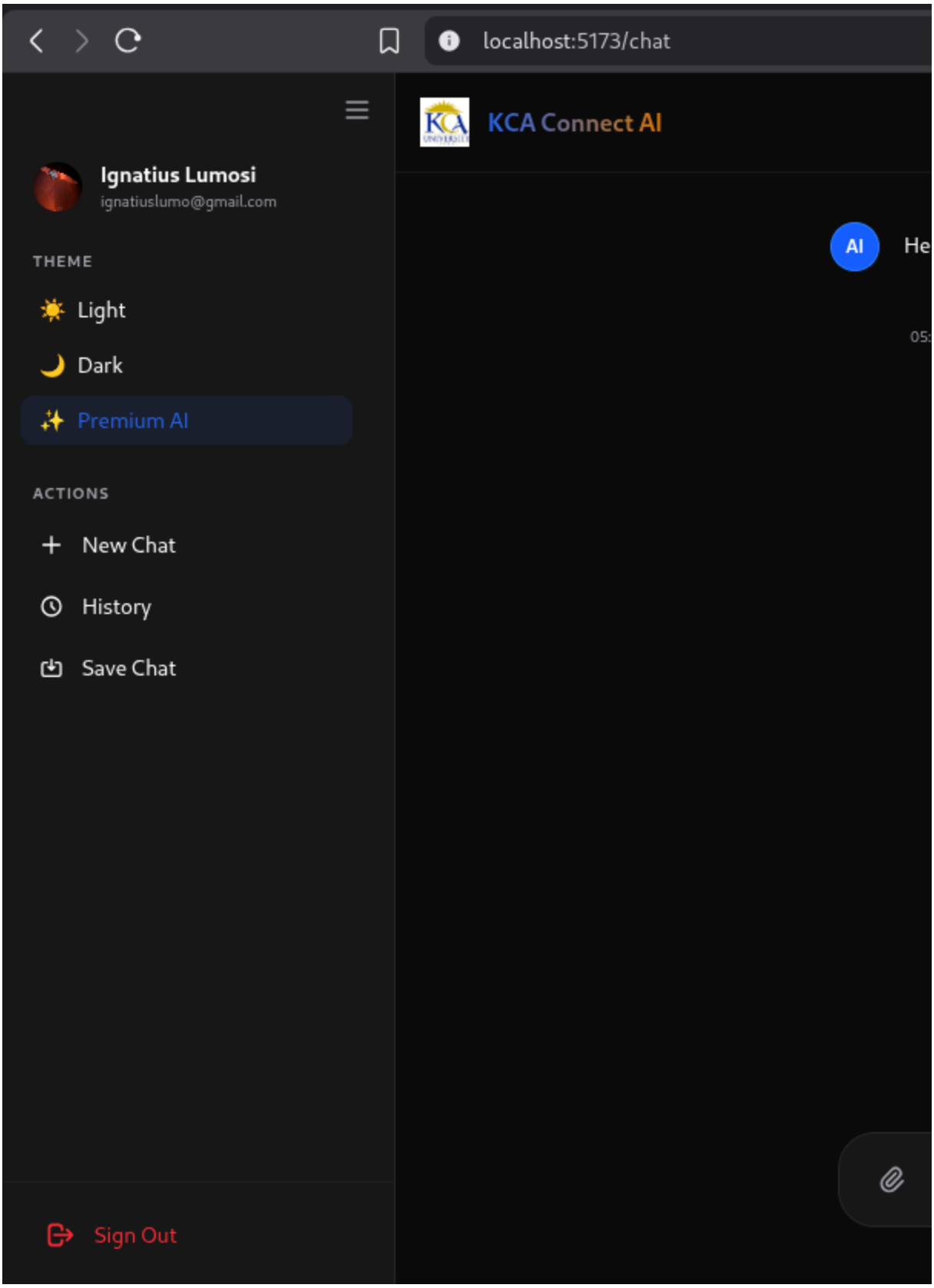
You can continue the conversation from where you left off.

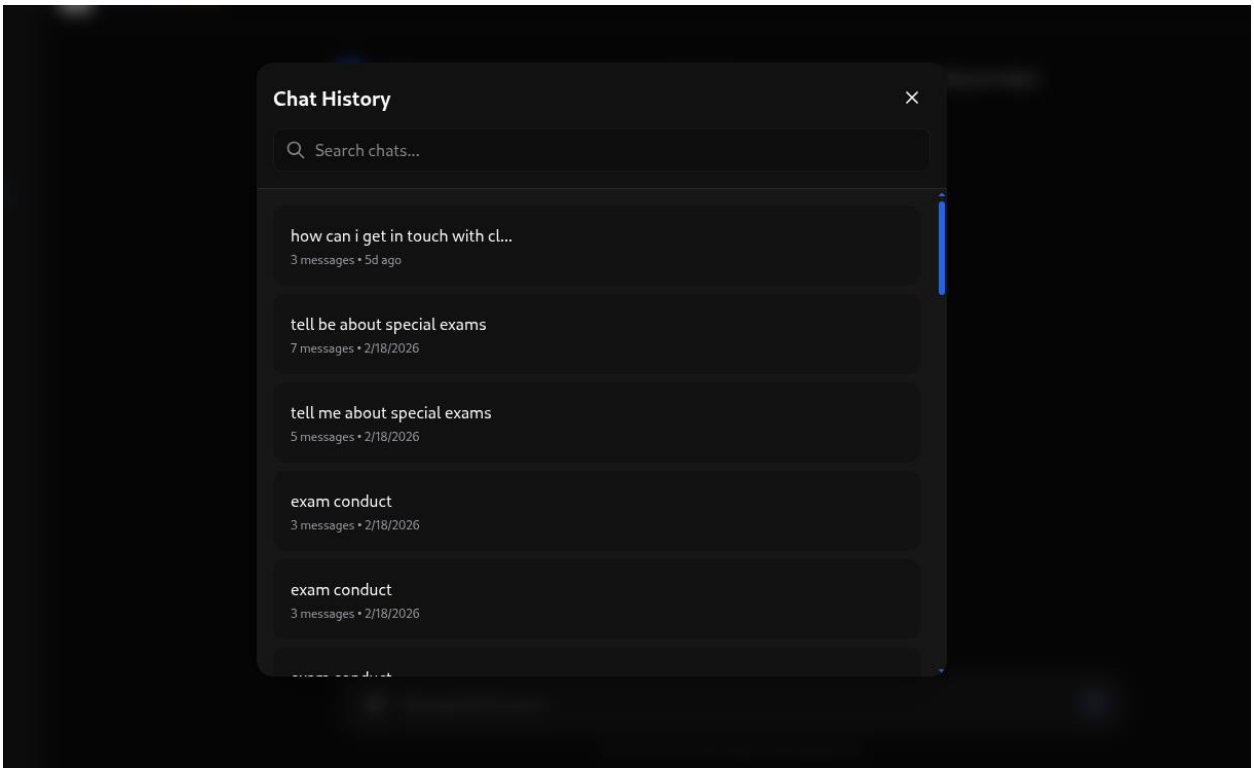
Deleting a chat:

Open Chat History.

Click the Delete icon next to the chat.

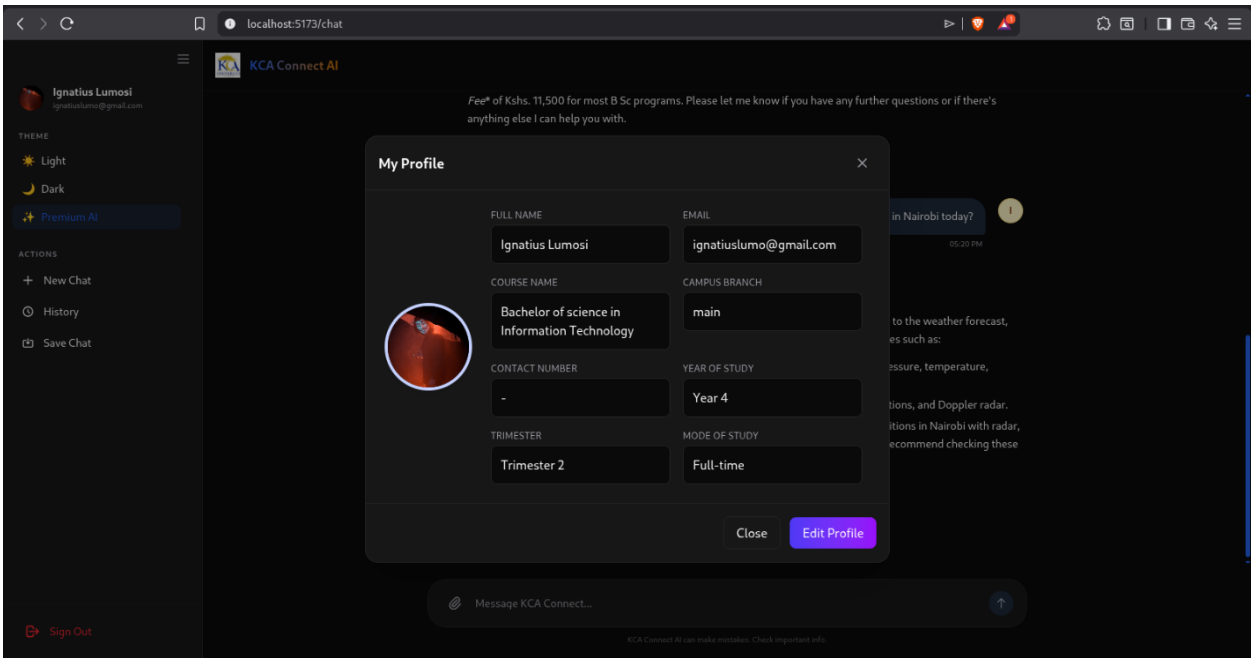
Confirm the deletion.


localhost:5173/chat
KCA Connect AI
Ignatius Lumosi
ignatiuslumo@gmail.com
THEME
Light
Dark
Premium AI
ACTIONS
New Chat
History
Save Chat
Sign Out
AI
He


Chat History
Search chats...
how can i get in touch with cl...
3 messages • 5d ago
tell be about special exams
7 messages • 2/18/2026
tell me about special exams
5 messages • 2/18/2026
exam conduct
3 messages • 2/18/2026
exam conduct
3 messages • 2/18/2026

E. Managing Your Profile

1. Click the Profile icon ( ) in the sidebar.
2. Your current profile information is displayed.



3. Click "Edit Profile" to modify:

- Full Name
- Campus Branch
- Year of Study (for students)
- Trimester
- Mode of Study (Full-time / Part-time)

4. Click "Save Changes" to persist updates.

Note: Email and Role cannot be changed after registration. Contact an administrator if changes are needed.

Profile-aware responses: The AI uses your profile (campus, year, trimester) to provide personalised answers such as campus-specific timetables.

7. CONCLUSIONS AND RECOMMENDATIONS

7.1 Project Conclusions

The KCA Connect Agentic AI project successfully demonstrated that agentic AI, when grounded in institutional data, can revolutionize university communication. The system met all its core objectives, providing a scalable, accurate, and user-friendly platform that significantly reduces administrative friction for KCA University.

7.2 Academic Contributions

This project contributed a unique implementation of a **resilient multi-LLM agent** specifically for the Kenyan academic context. It showcased how open-source embedding models can be leveraged to build production-grade RAG systems with minimal infrastructure costs.

7.3 Recommendations for Future Work

While the current system is robust, future iterations should focus on:

1. Multilingual Support: Extending the LLM prompts to support Swahili queries natively.

2. Voice Integration: Integrating browser-based speech-to-text for hands-free queries.

3. Native Mobile Support: Developing dedicated iOS/Android apps for enhanced push notifications and offline caching.

4. Automated Document Syncing: Building direct API connectors to university document repositories for real-time knowledge base updates.

REFERENCES

Deakin University. (2018). Genie digital assistant: Transforming student experience with AI. Retrieved from [https://www.deakin.edu.au]

Holmes, W., Bialik, M., & Fadel, C. (2019). Artificial Intelligence in Education: Promises and Implications for Teaching and Learning. Boston, MA: Centre for Curriculum Redesign.

Page, L. C., & Gehlbach, H. (2017). How Georgia State University used an AI chatbot to help students navigate enrolment. Education Next, 17(4), 68–74.

Lewis, P., et al. (2020). *Retrieval-Augmented Generation*.

Qdrant. (2023). *Vector Search Documentation*.

Supabase. (2024). *Open-source Backend Documentation*.

APPENDICES

Glossary

Term	Definition
AI (Artificial Intelligence)	Computer systems that simulate human intelligence, used here for natural language understanding and generation
API (Application Programming Interface)	A set of rules and endpoints allowing software components to communicate with each other
Cerebras	A third-party LLM provider used as an alternative AI backend
Embedding	A numerical vector representation of text used for semantic search
FastAPI	A high-performance Python web framework used for the KCA Connect backend
Gemini	Google's large language model used for generating AI responses
Glassmorphism	A modern UI design style using frosted-glass effects and translucency
Groq	A high-speed LLM inference provider used as the default AI backend
HuggingFace	An open-source AI platform providing the `all-MiniLM-L6-v2` embedding model
LangChain	A Python framework for building LLM-powered applications, used in the RAG pipeline
LLM (Large Language Model)	An AI model trained on large datasets to understand and generate human language
Markdown	A lightweight text formatting syntax used in AI responses (bold, lists, headers, etc.)
Qdrant	A high-performance vector database used to store and search document embeddings
RAG (Retrieval-Augmented Generation)	A technique combining information retrieval from a knowledge base with LLM generation for accurate, context-aware responses
React	A JavaScript library used to build the KCA Connect frontend user interface

Streaming	Word-by-word delivery of AI responses in real time, creating a typing effect
Supabase	An open-source backend-as-a-service platform providing authentication and PostgreSQL database
UAT (User Acceptance Testing)	Testing conducted with real users to validate the system meets their requirements
Vite	A fast JavaScript build tool used in the KCA Connect frontend
Vector Search	Searching for semantically similar text using numerical vector representations rather than exact keyword matching